

4-Week Backend Interview Revision Plan

Java (8–25, concurrency, exceptions, JVM memory), Design Patterns, Databases, Messaging Queues, Cache, Spring Boot, Elasticsearch — weekdays for theory, weekends for System Design theory + practice questions.

Week 1 — Core Java Deep Dive

Focus: language fundamentals interviewers probe hardest — concurrency, exceptions, and memory.

Day	Topic	Focus Areas
Mon	Java 8	Lambdas, Streams API, Optional, functional interfaces, default/static interface methods
Tue	Exception Handling	Checked vs unchecked exceptions, try-with-resources, custom exceptions, exception chaining, best practices
Wed	Concurrency & Multithreading I	Thread lifecycle, Runnable/Callable, synchronized, wait/notify, volatile, thread safety
Thu	Concurrency & Multithreading II	Executor framework, thread pools, CompletableFuture, java.util.concurrent (locks, atomics, ConcurrentHashMap), deadlocks/livelocks
Fri	JVM Memory Management	Heap vs stack, memory areas, GC algorithms (G1, ZGC), GC tuning basics, memory leaks, object lifecycle
Sat	System Design + Question	Theory: Scalability basics, load balancing, CAP theorem Question: Design a URL Shortener
Sun	System Design + Question	Theory: Consistent hashing, sharding, replication Question: Design a Rate Limiter

Week 2 — Newer Java Versions + Design Patterns

Focus: modern language features (lighter depth) and the classic GoF pattern catalog.

Day	Topic	Focus Areas
Mon	Java Collections Framework	ArrayList vs LinkedList, HashMap/TreeMap/LinkedHashMap internals, HashSet/TreeSet, Comparable vs Comparator, fail-fast vs fail-safe iterators
Tue	Java 9–25	Modules (JPMS), var, records, sealed classes, pattern matching; virtual threads (Loom), structured concurrency, scoped values, sequenced collections
Wed	Design Patterns – Creational	Singleton, Factory, Abstract Factory, Builder, Prototype
Thu	Design Patterns – Structural	Adapter, Decorator, Proxy, Facade, Composite
Fri	Design Patterns – Behavioral	Observer, Strategy, Command, State, Template Method, Chain of Responsibility
Sat	System Design + Question	Theory: Database sharding/partitioning strategies, replication lag Question: Design Instagram/Twitter Feed
Sun	System Design + Question	Theory: Caching strategies at scale, CDN basics Question: Design a Distributed Cache (Redis-like)

Week 3 — Databases + Messaging Queues

Focus: data storage tradeoffs and asynchronous, event-driven communication.

Day	Topic	Focus Areas
Mon	Database – Relational I	Normalization, indexing (B-tree/hash), transactions, ACID, isolation levels
Tue	Database – Relational II	Joins, query optimization, execution plans, connection pooling
Wed	Database – NoSQL	Document/Key-Value/Column/Graph stores, CAP tradeoffs, SQL vs NoSQL decision criteria
Thu	Messaging Queues I	Kafka architecture: partitions, consumer groups, offsets, brokers, replication
Fri	Messaging Queues II	Delivery guarantees (at-least-once/exactly-once), idempotency, dead-letter queues, Kafka vs RabbitMQ
Sat	System Design + Question	Theory: Message-queue-based architectures, event-driven design Question: Design a Notification System
Sun	System Design + Question	Theory: Search system design fundamentals Question: Design a Distributed Logging/Monitoring System

Week 4 — Cache, Spring Boot, Elasticsearch + Mock Interviews

Focus: caching internals, framework internals, search, then full mock system-design rounds.

Day	Topic	Focus Areas
Mon	Cache I	Redis data structures, eviction policies (LRU/LFU), TTL, persistence (RDB/AOF)
Tue	Cache II	Cache-aside, write-through/write-behind, cache stampede, distributed cache consistency
Wed	Spring Boot I	Auto-configuration, IoC/DI internals, bean lifecycle, application context
Thu	Spring Boot II	Spring MVC request lifecycle, AOP, transaction management (@Transactional internals)
Fri	Elasticsearch	Inverted index, sharding/replicas, query DSL basics, when to use ES vs a database
Sat	System Design + Question	Theory: Full mock interview structure & tradeoff discussion Question: Design a Ride-Sharing App (Uber)
Sun	System Design + Question	Theory: Full mock interview, wrap-up review of weak areas Question: Design a Chat System (WhatsApp)

Note: Each weekday slot is theory revision only; each Saturday and Sunday pairs a System Design theory topic with one matching practice question to apply it.